

Day03-文件权限与进程管理

Day 03 - 文件权限与进程管理

课程目标

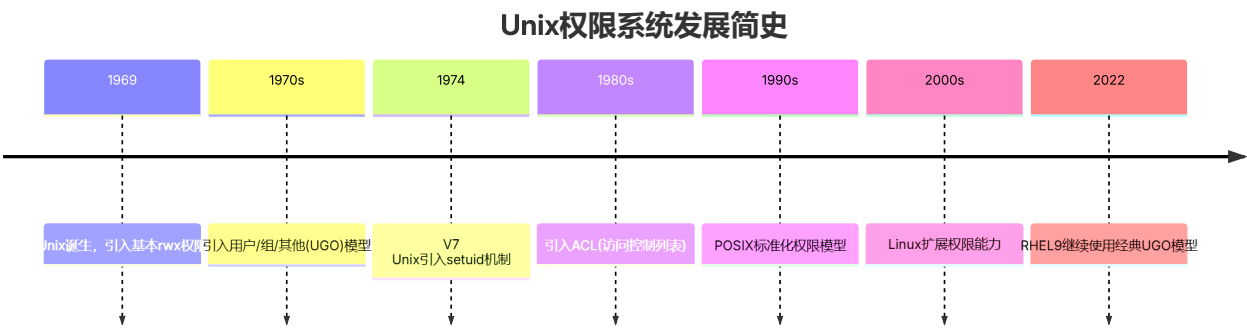
- 理解Linux文件系统权限模型
- 掌握chmod和chown命令的使用
- 学会设置特殊权限和默认权限
- 理解进程的状态和生命周期
- 掌握进程监控和管理命令
- 学会使用作业控制

上午 - 文件权限管理

课前故事：Unix权限的诞生

1969年，当Ken Thompson和Dennis Ritchie在贝尔实验室开发Unix时，他们面临一个根本性问题：在多用户系统中，如何防止用户互相干扰或窃取数据？

当时的解决方案诞生了**Unix权限模型**——一个简洁而强大的安全体系。



为什么**Unix**权限如此优雅？

Unix权限系统遵循四个核心设计原则：

1. 简单性：只有3种基本权限（rwx）
 - r (read) ： 读取权限
 - w (write) ： 写入权限
 - x (execute) ： 执行权限
2. 清晰性：只有3个主体类别
 - User (用户/所有者)

- Group (组)
 - Other (其他人)
3. 灵活性：通过组合实现复杂需求
- 权限可以针对不同类别分别设置
 - 支持权限掩码和特殊权限位

核心理念："一切皆文件" + "简单权限" = 强大而安全的系统

⚠ 重要性：

- 文件权限是RHCSA必考内容
- 实际工作中常要求创建协作目录、设置特定权限
- 理解权限对系统安全至关重要

第一部分：文件权限基础

1.1 权限类型详解

三种基本权限 (必记)：

权限	字母	数值	对文件的影响	对目录的影响	重要性
读取	r	4	可以读取文件内容	可以列出目录内容(ls)	☆☆☆
写入	w	2	可以更改文件内容	可以创建或删除文件	☆☆☆
执行	x	1	可以作为命令执行	可以cd进入目录	☆☆☆

权限的深层含义：

SHELL

读取权限(r)

对于文件: cat file.txt → 可以查看内容

对于目录: ls /directory → 可以列出文件列表

写入权限(w)

对于文件: echo "text" > file → 可以修改内容

对于目录: touch /dir/newfile → 可以创建新文件

rm /dir/oldfile → 可以删除文件

执行权限(x)

对于文件: ./script.sh → 可以运行脚本

对于目录: cd /directory → 可以进入目录

⚠ 易错点：目录权限的陷阱

X 常见误解
"如果我对目录有w权限，我就可以修改目录中的文件"

✔ 正确理解
目录的w权限只允许：
- 在目录中创建新文件
- 删除目录中的文件
- 重命名目录中的文件

但不能：
- 修改已有文件的内容（需要文件本身的w权限）
- 读取文件的内容（需要文件本身的r权限）

权限组合速查表：

权限组合	数值	符号	说明	常见用途
rwX	7	完全权限	读写执行	可执行脚本、程序
rw-	6	读写	不能执行	配置文件、数据文件
r-X	5	读执行	只读可执行	系统命令、库文件
r--	4	只读	只能读取	只读配置、文档
--X	1	仅执行	只能执行	特殊场景（很少用）
---	0	无权限	完全禁止	敏感文件

1.2 权限显示深度解析

查看文件权限（完全解析）：

```
# 查看文件详细信息
ls -l /etc/passwd

# 输出示例：
-rw-r--r--. 1 root root 1980 Jan 15 10:00 /etc/passwd

# 完整解析（10位权限字符串）：
| | | | | | | | | |
| | | | | | | | | | 其他用户权限（r--）
| | | | | | | | | | 组权限（r--）
| | | | | | | | | | 所有者权限（rw-）
| | | | | | | | | | 特殊权限位（. 表示无特殊权限）
| | | | | | | | | | 文件类型（- = 普通文件）
| | | | | | | | | | setuid/setgid/sticky位（s/S/t/T）
```

文件类型（第1位）：

类型	符号	说明	示例
普通文件	-	常规文件、脚本、文档	<code>file.txt</code>
目录	d	文件夹	<code>/home/user/</code>
符号链接	l	软链接	<code>/bin/ls -> ../bin/ls</code>
块设备	b	硬盘、U盘等	<code>/dev/sda</code>
字符设备	c	终端、键盘等	<code>/dev/tty</code>
套接字	s	进程间通信	<code>/var/run/docker.sock</code>
管道	p	命名管道	-

⚠ 提示：记住最常见的3种

- - (普通文件)
- d (目录)
- l (符号链接)

权限匹配详解（必须理解）：

Linux权限匹配顺序（重要！）

当用户尝试访问文件时：

第1步：检查用户是否是文件所有者

- YES → 使用所有者权限 → 匹配结束
- NO → 继续下一步

第2步：检查用户是否属于文件所属组

- YES → 使用组权限 → 匹配结束
- NO → 继续下一步

第3步：使用其他用户权限

- 无论是否匹配，都使用这个权限

关键点：

- 权限匹配是顺序执行的，一旦匹配成功就停止
- 第一个匹配的权限类别会被使用，不会继续检查后续类别
- 如果既不是所有者也不属于组，则使用other权限

⚠ 关键理解：

- 一旦匹配到某一类，就使用该类的权限
- 不是累加的！
- 优先级：所有者 > 组 > 其他

实战示例（必须掌握）：

SHELL

```
# 场景1: 文件权限为 -rw-r----- (640)
# 所有者: user1 (user1组)
# 所属组: group1
# 用户情况:
#   - user1 (所有者)           → rw- (可读写)
#   - user2 (group1成员)       → r-- (只读)
#   - user3 (不属于group1)     → --- (无权限)

# 场景2: 用户同时匹配多个类别
# 文件: -rw-rw---- (660)
# 所有者: user1 (wheel组)
# 所属组: wheel
# 访问用户: user1 (同时是所有者和wheel组成员)
# 结果: 使用所有者权限 rw- (不使用组权限!)
# 原因: 所有者优先级最高

# 场景3: 目录权限的陷阱
# 目录: drwxr-xrwx (755)
# 所有者: root
# 文件: -rw----- (600, 只有root可读)
# 普通用户能否读取该文件?
# 答: 不能! 虽然目录允许任何人进入(rx),
#     但文件本身拒绝其他人读取(---)
```

🚨 场景分析：

SHELL

```
# 题目: user1对file.txt有什么权限?
# 条件: file.txt权限为 -rw-r--r--, 所有者为root, 组为root
# user1不是root, 也不是root组成员
# 分析: user1是"其他用户" → 使用其他用户权限 r--
# 答案: user1可以读取文件内容

# 题目: user2能否删除/data/directory下的文件?
# 条件: /data/directory权限为 drwxrwxrwx (777)
#       file.txt权限为 -rw----- (600), 所有者为root
#       user2不是root
# 分析: 用户对目录有w权限, 可以删除目录中的文件
#       (即使他不能读取文件内容!)
# 答案: 可以删除
```

选择练习

练习名称	对应章节	说明
选择练习：解释文件系统权限	第07章 第01节	理解Linux文件权限概念

第二部分：修改权限(chmod)

课前故事：chmod命令的诞生

chmod (change mode) 是Unix最古老也最重要的命令之一。在1970年代初期，Unix设计师们需要一个简单的方法来控制文件访问。

为什么叫"mode"（模式）？

因为在Unix哲学中，文件的访问模式由一组标志位控制：

- 每个权限是一个二进制位
- $r = 100$ (二进制), $w = 010$, $x = 001$
- 组合起来就形成了"模式"

符号法	数值法
符号法 <pre>chmod [who][operator][permissions]</pre> who (对象): • u = user (所有者) • g = group (所属组) • o = others (其他用户) • a = all (所有人) operator (操作符): • + = 添加权限 • - = 删除权限 • = = 精确设置 permissions: • r = read • w = write • x = execute 示例: <pre>chmod u+x file chmod go+r file</pre>	数值法 <pre>chmod [数字] file</pre> 权限值: - r (read) = 4 - w (write) = 2 - x (execute) = 1 组合: $rw\text{-}x = 4+2+1 = 7$ $rw\text{-} = 4+2+0 = 6$ $r\text{-}x = 4+0+1 = 5$ $r\text{-} = 4+0+0 = 4$ 示例: <pre>chmod 755 file (rwxr-xr-x)</pre> <pre>chmod 644 file (rw-r--r--)</pre> 适用场景: 快速精确设置权限

💡 两种方法都要掌握!
符号法: 适合相对修改 (添加/删除特定权限)
数值法: 适合精确设置 (常用于快速配置)

⚠️ 技巧：两种方法都要掌握！

- 符号法：适合相对修改（添加/删除特定权限）
- 数值法：适合精确设置（常用于快速配置）

2.1 chmod - 符号法（直观灵活）

符号法完整语法：

```
chmod [who][operator][permissions] file
```

```
# who (对象) :
#   u      = user (所有者)
#   g      = group (所属组)
#   o      = others (其他用户)
#   a      = all (所有人, ugo之和)

# operator (操作符) :
#   +      = 添加权限
#   -      = 删除权限
#   =      = 精确设置 (覆盖原有权限)

# permissions (权限) :
#   r      = read (读)
#   w      = write (写)
#   x      = execute (执行)
```

符号法示例 (常用) :

```
# 示例1: 为所有者添加执行权限
chmod u+x script.sh
# 原来: rw-r--r-- (644)
# 现在: rwxr--r-- (745)
# 只修改了所有者权限, 其他不变

# 示例2: 为组和其他用户添加读权限
chmod go+r file.txt
# 原来: rw----- (600)
# 现在: rw-r--r-- (644)
# g = group, o = others, 同时添加

# 示例3: 删除其他用户的写权限
chmod o-w file.txt
# 原来: rw-rw-rw- (666)
# 现在: rw-rw-r-- (664)
# 只删除其他用户的w权限

# 示例4: 删除所有执行权限
chmod a-x script.sh
# 原来: rwxr-xr-x (755)
# 现在: rw-r--r-- (644)
# a = all, 删除u、g、o的x权限

# 示例5: 精确设置权限
chmod u=rwx,g=rx,o=x directory/
# 不管原来什么权限, 设置为:
# 所有者: rwx
# 组: rx
# 其他: x
# 结果: rwxr-x--x (751)
```

⚠ 易错点：

SHELL

```
# X 错误理解
chmod u+x,g+w file.txt
# 这是错的！不能一次指定多个操作符

# ✓ 正确做法
chmod u+x,g+w file.txt    # 用逗号分隔多个操作
chmod go+rw file.txt      # 或者组合对象

# X 危险操作
chmod -R 777 /
# 删除系统所有权限保护！千万不要这样做！

# ✓ 正确的递归操作
chmod -R 755 /var/www/html
# 只修改特定目录
```

2.2 chmod - 数值法（快速精确）

权限数值详解：

权限数值计算原理（二进制基础）

```
rwX = r(4) + w(2) + x(1) = 7
r-- = r(4) + w(0) + x(0) = 4
-w- = r(0) + w(2) + x(0) = 2
--x = r(0) + w(0) + x(1) = 1
```

二进制表示：

```
rwX = 111（二进制）= 7（十进制）
rW- = 110（二进制）= 6（十进制）
r-x = 101（二进制）= 5（十进制）
r-- = 100（二进制）= 4（十进制）
```

完整权限值表（必背）：

权限	二进制	数值	符号表示	常见用途	使用频率
无权限	000	0	---	不使用	★
仅执行	001	1	--x	极少用	★
仅写	010	2	-w-	极少用	★
写执行	011	3	-wx	极少用	★
仅读	100	4	r--	只读文件	★★★★
读执行	101	5	r-x	命令/脚本	★★★★
读写	110	6	rw-	配置文件	★★★★

权限	二进制	数值	符号表示	常见用途	使用频率
完全	111	7	rwX	所有权限	★★★★

常见权限值（必记）：

数值	符号	文件用途	目录用途	使用场景
644	rw-r--r--	配置文件	-	标准配置文件
755	rwXr-Xr-X	可执行文件	标准目录	脚本、程序
600	rw-----	敏感文件	-	私密数据、密钥
700	rwX-----	私有脚本	私有目录	用户主目录
640	rw-r-----	配置文件	-	组可读配置
750	rwXr-X---	-	协作目录	组协作目录
775	rwXrwxr-X	-	共享目录	团队共享
777	rwXrwxrwx	-	公共目录	临时目录

数值法示例（完全输出）：

```

# 示例1: 标准配置文件权限
chmod 644 /etc/httpd/conf/httpd.conf
# 设置为: rW-r--r--
# 所有者: 读写
# 组: 只读
# 其他: 只读

# 验证:
[root@servera ~]# ls -l /etc/httpd/conf/httpd.conf
-rw-r--r--. 1 root root 11753 Jan 15 10:00 /etc/httpd/conf/httpd.conf

# 示例2: 可执行脚本权限
chmod 755 /root/backup.sh
# 设置为: rwxr-xr-x
# 所有者: 读写执行
# 组: 读执行
# 其他: 读执行

# 示例3: 敏感文件权限
chmod 600 /root/.ssh/id_rsa
# 设置为: rW-----
# 只有root可以读写
# 其他用户完全无权限

# 示例4: 协作目录权限 (常见)
chmod 2770 /home/managers
# 注意: 2770 = 2(SGID) + 770(基本权限)
# 实际权限: rwxrwx---
# 设置SGID: 新文件继承目录组

# 验证SGID:
[root@servera ~]# ls -ld /home/managers
drwxrws---. 2 root sysmgrs 4096 Jan 15 10:30 /home/managers
#
#          ↑
#          s表示SGID已设置

```

⚠ 技巧: 快速计算权限值

心算方法: (r|w|x 读|写|执行 4|2|1)

- 需要rwx → 7
- 需要rW- → 6
- 需要r-x → 5
- 需要r-- → 4
- 需要--x → 1
- 无权限 ----→ 0

示例: 题目要求"所有者读写执行, 组读执行, 其他无权限"

所有者: rwx = 7

组: rx = 5

其他: --- = 0

答案: chmod 750 file.txt

2.3 递归修改权限 (-R选项)

递归修改的威力与危险：

```
# 基本语法
chmod -R [权限] [目录]

# -R = Recursive (递归)
# 修改目录及其下所有文件和子目录的权限
```

SHELL

递归修改示例：

```
# 示例1：设置网站目录权限（常见任务）
chmod -R 755 /var/www/html
# 递归设置html目录及所有子目录和文件为755

# 问题：文件不需要执行权限x！
# 更好的做法：区分文件和目录
find /var/www/html -type d -exec chmod 755 {} \;
find /var/www/html -type f -exec chmod 644 {} \;
# 目录：755 (rwxr-xr-x)
# 文件：644 (rw-r--r--)

# 示例2：设置用户家目录权限
chmod -R 700 /home/student
# 设置student家目录及所有内容为私有

# 验证：
[root@servera ~]# ls -ld /home/student
drwx-----. 5 student student 4096 Jan 15 10:30 /home/student
#           ↑
#           只有student有权限

# 示例3：危险的递归操作（X 绝对不要做）
chmod -R 777 /
# 删除整个系统的权限保护！
# 这会让任何用户都能修改系统文件
# 系统安全彻底崩溃！
```

SHELL

⚠ 使用场景：协作目录设置

```
# 题目：创建协作目录/home/managers
# 要求：sysmgrs组成员可以读写执行，其他人无权限
#       新创建的文件自动继承组权限

mkdir /home/managers
chown :sysmgrs /home/managers
chmod 2770 /home/managers

# 解释：
# 2 = SGID，新文件继承目录组
# 770 = rwxrwx---
#   所有者：rwx
#   组：rwx
#   其他：---

# 验证：
[root@servera ~]# ls -ld /home/managers
drwxrws---. 2 root sysmgrs 4096 Jan 15 10:30 /home/managers
#           ↑
#           s = SGID + execute

# 测试：
[student@node1 ~]$ cd /home/managers
[student@node1 managers]$ touch test.txt
[student@node1 managers]$ ls -l test.txt
-rw-rw-r--. 1 student sysmgrs 0 Jan 15 10:35 test.txt
#           ↑       ↑
#           文件属于sysmgrs组
```

指导练习：

练习名称	对应章节	说明
练习：从命令行管理文件系统权限	第07章 第02节	使用chmod修改文件权限

第三部分：修改所有权(chown)

chown的哲学：文件归属权

chown (change owner) 是管理文件所有权的核心命令。在Unix系统中，每个文件都有明确的归属：

- 所有者(user)

- 所属组(group)

为什么需要chown?

场景：用户 alice 创建了一个项目脚本，需要移交给用户 bob 接手维护

问题：文件所有者是 alice, bob 没有写入权限，无法修改

解决：chown bob:bob project.sh (或 chown bob:team project.sh 让团队组也可编辑)

结果：bob 成为文件所有者，可以正常修改维护

注意：root 用户可以修改任何文件，无需 chown。chown 主要用于普通用户/服务之间的文件所有权转移。

3.1 chown基础（必须掌握）

chown完整语法：

```
chown [选项] [所有者][:[组]] 文件
```

SHELL

基本用法：

```
chown user file.txt # 只修改所有者
```

```
chown user:group file.txt # 同时修改所有者和组
```

```
chown user: file.txt # 修改所有者，组设置为用户的登录组
```

```
chown :group file.txt # 只修改组（所有者不变）
```

chown示例详解：

```
# 示例1: 修改文件所有者
chown student /tmp/file.txt
# 文件所有者从root改为student
# 所属组保持不变

# 验证:
[root@servera ~]# ls -l /tmp/file.txt
-rw-r--r--. 1 student root 1024 Jan 15 10:00 /tmp/file.txt
#           ↑      ↑
#           所有者  所属组

# 示例2: 同时修改所有者和组
chown student:students /tmp/file.txt
# 所有者: student
# 所属组: students

# 验证:
[root@servera ~]# ls -l /tmp/file.txt
-rw-r--r--. 1 student students 1024 Jan 15 10:00 /tmp/file.txt
#           ↑      ↑
#           所有者  所属组

# 示例3: 只修改组
chown :wheel /tmp/file.txt
# 冒号前没有用户名 = 只修改组

# 验证:
[root@servera ~]# ls -l /tmp/file.txt
-rw-r--r--. 1 student wheel 1024 Jan 15 10:00 /tmp/file.txt
#           ↑      ↑
#           所有者  所属组 (改为wheel)

# 示例4: 使用UID和GID (高级)
chown 1000:1000 /tmp/file.txt
# 使用UID和GID而不是用户名/组名
# 1000:1000 = UID为1000的用户, GID为1000的组
```

⚠ 易错点: 冒号的含义

```
chown user file.txt      # 只修改所有者
chown user: file.txt     # 修改所有者, 组改为user的登录组
chown user:group file.txt # 修改所有者和组
chown :group file.txt    # 只修改组

# 记忆技巧:
# 冒号前 = 所有者
# 冒号后 = 组
# 只有冒号 = 只有组
```

3.2 chown递归修改(-R)

递归修改所有权：

基本语法

```
chown -R [所有者][:组] [目录]
```

-R = Recursive (递归)

修改目录及所有内容的所有权

SHELL

递归示例：

示例1: 移交整个目录 (常见)

```
chown -R student:student /home/student/
```

将student家目录及所有内容的所有权和组改为student

示例2: 修改网站目录所有权

```
chown -R apache:apache /var/www/html
```

将网站目录及所有文件改为apache用户和组

常用于设置Web服务器权限

验证递归结果：

```
[root@servera ~]# ls -lR /var/www/html | head -20
/var/www/html:
```

total 4

```
-rw-r--r--. 1 apache apache 1024 Jan 15 10:00 index.html
```

```
-rw-r--r--. 1 apache apache 2048 Jan 15 09:30 about.html
```

#

↑ ↑

#

所有文件都是apache:apache

示例3: 只修改组, 保留所有者

```
chown -R :wheel /shared/directory
```

递归修改目录及所有内容的组为wheel

所有者保持不变

SHELL

⚠ 场景分析：

```
# 题目: /data/project目录属于root, 要求:
# 1. 将所有权移交给user1
# 2. 所属组设置为developers
# 3. 递归应用到所有子目录和文件

chown -R user1:developers /data/project

# 验证:
[root@servera ~]# ls -ld /data/project
drwxr-xr-x. 5 user1 developers 4096 Jan 15 10:30 /data/project

# 检查子目录:
[root@servera ~]# ls -lR /data/project | grep "^d"
drwxr-xr-x. 2 user1 developers 4096 Jan 15 10:30 src
drwxr-xr-x. 2 user1 developers 4096 Jan 15 10:30 doc
# 所有子目录和文件的所有权都已修改
```

3.3 chgrp - 专用于修改组

chgrp (change group) 简介:

chgrp是专门用于修改文件所属组的命令, 功能等同于 **chown :group**

```
# 基本语法
chgrp [选项] 组 文件

# 示例:
chgrp wheel file.txt          # 修改文件的组
chgrp -R developers /project  # 递归修改目录的组
```

chgrp vs chown :group:

```
# 方法1: 使用chown
chown :wheel file.txt

# 方法2: 使用chgrp
chgrp wheel file.txt

# 结果: 完全相同
# 选择: chown更通用, chgrp更专一

# 为什么有chgrp?
# 1. 历史原因: chgrp出现得更早
# 2. 语义清晰: 一看就知道是修改组
# 3. 惯例使用: 许多老脚本仍在使用
```


相关练习：指导练习

练习名称	对应章节	说明
练习：从命令行管理文件系统权限	第07章 第02节	使用chown修改文件所有权

第四部分：特殊权限详解

课前故事：特殊权限的诞生

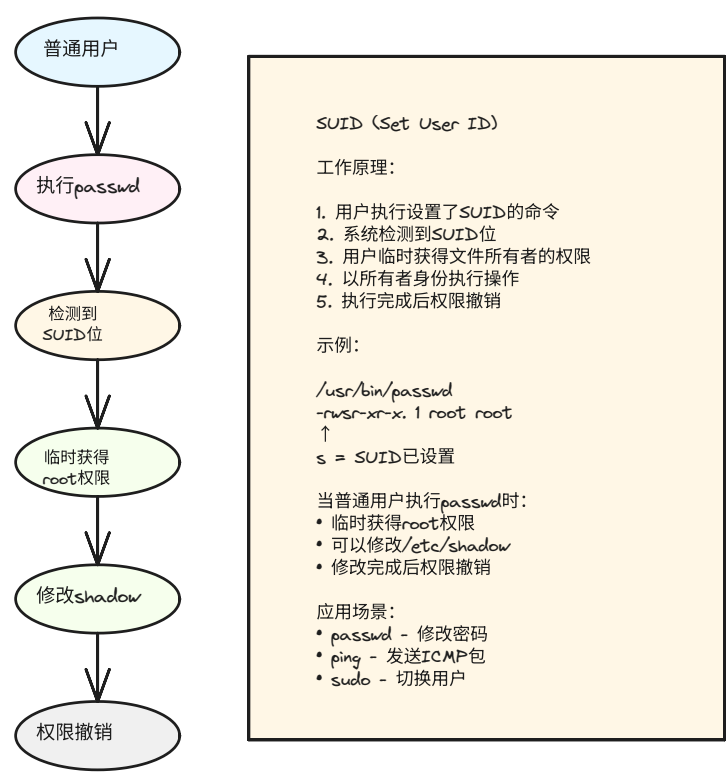
1974年，Unix Version 7发布时，引入了一个革命性的概念：**SUID (Set User ID)**。

问题背景：

- 普通用户需要修改密码（存储在/etc/shadow）
- /etc/shadow只有root可写
- 如何让普通用户安全地修改密码？

解决方案：**SUID**

SUID 工作原理



SUID的智慧：

- 用户执行passwd命令

- passwd所有者是root
- passwd设置了SUID
- 用户临时获得root权限
- 成功修改/etc/shadow
- 执行结束，权限撤销

后来扩展的另外两种特殊权限：

- **SGID (1980s)**：解决组协作问题
- **Sticky Bit (1980s)**：解决共享目录问题

4.1 特殊权限完整对比

三种特殊权限对照表（必背）：

权限	名称	符号设置	数值	对文件	对目录	重要性
SUID	Set User ID	u+s	4	以所有者身份执行	无影响	☆☆☆
SGID	Set Group ID	g+s	2	以所属组身份执行	新文件继承组	☆☆☆
Sticky	Sticky Bit	o+t	1	无影响	只有所有者可删除	☆☆

⚠ 权限显示字符含义（易考）：

- **S** = SUID + 无执行权限
- **s** = SUID + 有执行权限
- **T** = Sticky + 无执行权限
- **t** = Sticky + 有执行权限

SHELL

示例对比：

```
-rwsr-xr-x # s = SUID + x, 有执行权限
-rwSr-xr-x # S = SUID + 无x, 无执行权限
drwxrwxrwt # t = Sticky + x, 有执行权限
drwxrwxrWT # T = Sticky + 无x, 无执行权限
```

4.2 SUID深度解析

SUID (Set User ID) 工作原理：

正常情况:

user1 运行 /bin/ls → user1权限 → 列出user1能访问的文件

SUID情况:

user1 运行 /usr/bin/passwd → root权限 → 修改/etc/shadow

↑
passwd有SUID, 所有者是root

SUID的典型应用场景:

SHELL

场景1: passwd命令 (最经典示例)

ls -l /usr/bin/passwd

-rwsr-xr-x. 1 root root 35504 Jul 16 20:10 /usr/bin/passwd

↑

s表示SUID已设置

工作流程:

1. user1执行passwd命令
2. 系统检测到passwd有SUID位
3. user1临时获得root权限
4. passwd修改/etc/shadow
5. 执行完成, 权限撤销

场景2: ping命令

ls -l /usr/bin/ping

-rwsr-xr-x. 1 root root 81968 Jul 16 20:10 /usr/bin/ping

ping需要ICMP原始套接字 (需要root权限)

设置SUID后, 普通用户可以ping

场景3: sudo命令

ls -l /usr/bin/sudo

-rwsr-xr-x. 1 root root 165888 Jul 16 20:10 /usr/bin/sudo

sudo本身也需要SUID才能工作

设置SUID的方法:

```
# 方法1: 符号法
chmod u+s /path/to/file
# u+s = 为所有者(user)添加SUID权限

# 方法2: 数值法 (推荐使用)
chmod 4755 /path/to/file
# 4755 = 4(SUID) + 755(基本权限)
# 4 = SUID位
# 755 = rwxr-xr-x

# 示例:
chmod 4755 /usr/local/bin/myapp
# 设置后:
-rwsr-xr-x. 1 root root 4096 Jan 15 10:00 /usr/local/bin/myapp

# 移除SUID:
chmod u-s /usr/local/bin/myapp
# 或
chmod 0755 /usr/local/bin/myapp
```

⚠ 安全警告 (必知) :

```
# X 危险: 给脚本设置SUID
chmod 4755 /tmp/malicious.sh
# 如果这个脚本有漏洞, 攻击者可以获取root权限

# X 危险: 给编辑器设置SUID
chmod 4755 /usr/bin/vim
# 用户可以通过vim读取任何文件 (包括/etc/shadow)

# ✓ 安全原则:
# 1. 只给必要的程序设置SUID
# 2. 确保程序无安全漏洞
# 3. 定期审计SUID文件
find / -perm -4000 -type f 2>/dev/null
# 查找所有设置了SUID的文件
```

4.3 SGID深度解析

SGID (Set Group ID) 的两个用途:

SGID在文件上: 以文件所属组身份执行
SGID在目录上: 新创建的文件继承目录的组

SGID for Files (文件) :

```
# 示例：创建一个需要特定组权限的程序
groupadd developers
useradd -G developers alice
useradd -G developers bob

# 编写程序（简化示例）
cat > /usr/local/bin/teamtool << 'EOF'
#!/bin/bash
# 程序需要访问共享目录：/shared/teamdata
echo "Accessing shared resources..."
ls /shared/teamdata
EOF

# 设置程序所有者和组
chown root:developers /usr/local/bin/teamtool

# 设置SGID
chmod 2750 /usr/local/bin/teamtool
# 2 = SGID
# 750 = rwxr-x---
# 所有者(root): rwx
# 组(developers): r-x
# 其他: ---

# 验证：
ls -l /usr/local/bin/teamtool
-rwxr-s---. 1 root developers 123 Jan 15 10:00 /usr/local/bin/teamtool
#           ↑
#           s表示SGID已设置

# 工作原理：
# alice执行teamtool → 以developers组身份运行
# bob执行teamtool → 以developers组身份运行
# → 即使alice和bob的主组不是developers，也能访问共享资源
```

SGID for Directories（目录） - 重点：

```
# 场景：创建团队协作目录
mkdir -p /shared/project
groupadd developers

# 设置目录组
chown :developers /shared/project

# 设置SGID
chmod 2770 /shared/project
# 2 = SGID
# 770 = rwxrwx---
# 所有者: rwx
# 组: rwx
# 其他: ---

# 验证:
ls -ld /shared/project
drwxrws---. 2 root developers 4096 Jan 15 10:00 /shared/project
#           ↑
#           s表示SGID已设置

# 测试SGID效果:
su - alice
$ cd /shared/project
$ touch alice_file.txt
$ ls -l alice_file.txt
-rw-rw-r--. 1 alice developers 0 Jan 15 10:30 alice_file.txt
#           ↑      ↑
#           文件自动属于developers组!

su - bob
$ cd /shared/project
$ touch bob_file.txt
$ ls -l bob_file.txt
-rw-rw-r--. 1 bob developers 0 Jan 15 10:31 bob_file.txt
#           ↑      ↑
#           文件也属于developers组!

# 好处:
# 1. 所有团队成员创建的文件都属于同一个组
# 2. 团队成员可以互相编辑 (如果组权限允许)
# 3. 简化了文件权限管理
```

⚠ 使用场景 (必会) :

```
# 题目：创建协作目录/shared/marketing
# 要求：
# 1. marketing组可读写执行
# 2. 其他用户无任何权限
# 3. 新创建的文件自动继承组权限

# 答案：
mkdir /shared/marketing
groupadd marketing
chown :marketing /shared/marketing
chmod 2770 /shared/marketing

# 验证：
ls -ld /shared/marketing
drwxrws---. 2 root marketing 4096 Jan 15 10:00 /shared/marketing
#           ↑
#           s = SGID设置成功

# 测试：
[marketing_user@node1]$ cd /shared/marketing
[marketing_user@node1]$ touch test.txt
[marketing_user@node1]$ ls -l test.txt
-rw-rw-r--. 1 marketing_user marketing 0 Jan 15 10:05 test.txt
#                                     ↑
#                                     组继承成功
```

4.4 Sticky Bit深度解析

Sticky Bit（粘滞位）的设计初衷：

1970年代，在大学的多用户Unix系统中，`/tmp`目录是所有用户共享的临时文件存储区。

问题：

```
用户A创建了 /tmp/myfile.txt
用户B可以删除 /tmp/myfile.txt (因为/tmp对所有用户开放写权限)
→ 用户A的数据被恶意删除！
```

解决方案：Sticky Bit

```
# 查看/tmp的Sticky Bit
ls -ld /tmp
drwxrwxrwt. 39 root root 4096 Feb 8 20:52 /tmp
#           ↑
#           t表示Sticky Bit

# 说明:
- rwxrwxrwt = 所有用户都可以读写执行
- 但是: 只能删除自己拥有的文件
- Sticky Bit保护用户文件不被其他人删除
```

Sticky Bit工作原理:

没有Sticky Bit:

```
/tmp/drwxrwxrwx (777)
├─ user1_file.txt (user1所有)
└─ user2可以删除 user1_file.txt X
```

有Sticky Bit:

```
/tmp/drwxrwxrwt (1777)
├─ user1_file.txt (user1所有)
│   └─ 只有user1可以删除 ✓
└─ user2_file.txt (user2所有)
    └─ 只有user2可以删除 ✓
```

设置Sticky Bit:


```
# 方法1: 符号法
chmod o+t /shared/directory
# o+t = 为其他用户(other)添加Sticky Bit

# 方法2: 数值法 (推荐)
chmod 1777 /shared/directory
# 1777 = 1(Sticky) + 777(基本权限)
# 1 = Sticky Bit
# 777 = rwxrwxrwx

# 示例:
mkdir /tmp/public
chmod 1777 /tmp/public

# 验证:
ls -ld /tmp/public
drwxrwxrwt. 2 root root 4096 Jan 15 10:00 /tmp/public
#               ↑
#               t表示Sticky Bit

# 测试效果:
[user1@node1]$ echo "data1" > /tmp/public/user1.txt
[user2@node1]$ echo "data2" > /tmp/public/user2.txt
[user2@node1]$ rm /tmp/public/user1.txt
rm: cannot remove '/tmp/public/user1.txt': Operation not permitted
#               ↑
#               user2无法删除user1的文件!

[user1@node1]$ rm /tmp/public/user1.txt
# user1可以删除自己的文件 ✓
```

⚠ 使用场景 (必会) :

```
# 题目：创建共享目录/shared/uploads
# 要求：
# 1. 所有用户都可以上传文件
# 2. 所有用户都可以下载文件
# 3. 用户只能删除自己的文件

# 答案：
mkdir -p /shared/uploads
chmod 1777 /shared/uploads
# 1777 = 1(Sticky) + 777(rwxrwxrwx)

# 验证：
ls -ld /shared/uploads
drwxrwxrwt. 2 root root 4096 Jan 15 10:00 /shared/uploads

# 测试场景：
[user1@node1]$ echo "user1 data" > /shared/uploads/file1.txt
[user2@node1]$ echo "user2 data" > /shared/uploads/file2.txt

[user2@node1]$ cat /shared/uploads/file1.txt # 可以读取 √
[user2@node1]$ rm /shared/uploads/file1.txt # 不能删除 X
#                               ↑
#   Sticky Bit保护了user1的文件
```

三种特殊权限组合使用（高级）：

```
# 场景：完全协作目录
# 要求：
# 1. 团队成员可以创建文件
# 2. 新文件自动继承目录的组
# 3. 成员只能删除自己的文件

mkdir /shared/teamwork
groupadd teamdev
chown :teamdev /shared/teamwork
chmod 3777 /shared/teamwork
# 3777 = 1(Sticky) + 2(SGID) + 777(基本权限)

# 验证：
ls -ld /shared/teamwork
drwxrwsrwt. 2 root teamdev 4096 Jan 15 10:00 /shared/teamwork
#           ↑  ↑
#           s  t
#           SGID Sticky

# 效果：
# 1. s(SGID)：新文件属于teamdev组
# 2. t(Sticky)：只能删除自己的文件
# 3. 777：所有团队成员可以读写执行
```

指导练习：

练习名称	对应章节	说明
练习：管理默认权限和文件访问	第07章 第03节	管理特殊权限和文件访问控制

第五部分：默认权限(umask)

课前故事：umask的智慧

1970年代，Unix设计师们面临一个重要的安全决策：当用户创建新文件时，应该给予什么默认权限？

问题背景：

- 如果默认权限太开放（如777）：
- 用户可能无意中创建敏感文件

→ 其他用户可以访问这些文件

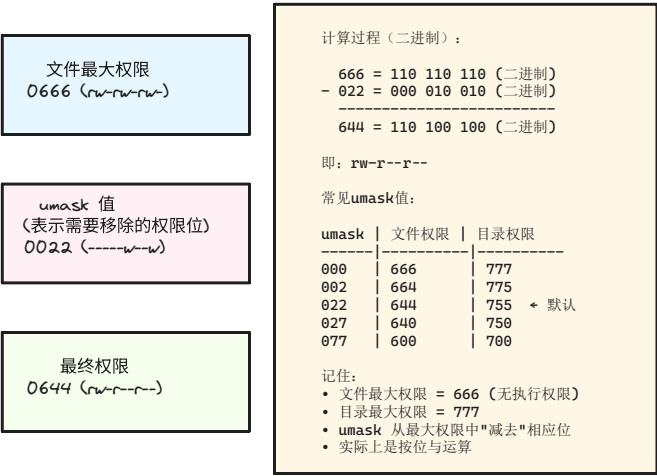
→ 安全隐患！
- 如果默认权限太严格（如600）：
- 协作变得困难

→ 每次都要手动修改权限

→ 用户体验差！

解决方案：umask（用户权限掩码）

umask 工作原理



umask的哲学：

- 默认拒绝 (Default Deny)
- 用户主动决定要开放什么权限
- 而不是系统强制给予特定权限

⚠ 重要性:

- 理解umask计算原理
- 知道常见umask值及其效果
- 能够预测新文件的默认权限

5.1 umask 工作原理详解

umask是什么?

umask (user mask) 控制新建文件和目录的默认权限

基本原理:

文件最大默认权限: 0666 (rw-rw-rw-)

目录最大默认权限: 0777 (rwxrwxrwx)

umask从最大权限中"减去"相应位

为什么文件最大权限是666而不是777?

SHELL

```
# 原因: 安全考虑
# 新创建的文件默认不应该有执行权限(x)
# 执行权限需要用户明确添加 (chmod +x)

# 理由:
# 1. 文本文件不需要执行权限
# 2. 防止意外执行恶意脚本
# 3. 用户必须主动设置执行权限 (显式意图)

# 目录为什么可以有执行权限?
# 目录的x权限表示可以cd进入
# 这是目录的基本功能, 默认允许
```

umask计算详解 (二进制方式) :

```
# 示例1: umask = 022
# 最大文件权限: 0666 (rw-rw-rw-)
# umask值:      0022 (---w--w-)
# 计算过程 (二进制) :
#   666 = 110 110 110 (二进制)
#   022 = 000 010 010 (二进制)
#   -----
#   644 = 110 100 100 (二进制) = rw-r--r--

# 最大目录权限: 0777 (rwxrwxrwx)
# umask值:      0022 (---w--w-)
# 计算过程:
#   777 = 111 111 111 (二进制)
#   022 = 000 010 010 (二进制)
#   -----
#   755 = 111 101 101 (二进制) = rwxr-xr-x

# 示例2: umask = 077
# 最大文件权限: 0666 (rw-rw-rw-)
# umask值:      0077 (----rwxrwx)
# 计算过程:
#   666 = 110 110 110 (二进制)
#   077 = 000 111 111 (二进制)
#   -----
#   600 = 110 000 000 (二进制) = rw-----

# 最大目录权限: 0777 (rwxrwxrwx)
# umask值:      0077 (----rwxrwx)
# 计算过程:
#   777 = 111 111 111 (二进制)
#   077 = 000 111 111 (二进制)
#   -----
#   700 = 111 000 000 (二进制) = rwx-----
```

⚠ 计算技巧 (必记) :

快速心算方法:

对于文件 (最大666) :

- umask的某位是0 → 权限保持6或4或2 (取决于其他位)
- umask的某位是2 → 减去2 (4→2, 6→4)
- umask的某位是4 → 减去4 (6→2, 4→0)
- umask的某位是6 → 减去6 (6→0)
- umask的某位是7 → 减去6 (文件不能有执行权限)

对于目录 (最大777) :

- 直接用 777 - umask值

常见umask值对照表 (必背) :

umask	文件权限	目录权限	说明	使用场景
000	rw-rw-rw- (666)	rwxrwxrwx (777)	完全开放	极少使用, 不安全
002	rw-rw-r-- (664)	rwxrwxr-x (775)	组可写	私有协作环境
022	rw-r--r-- (644)	rwxr-xr-x (755)	标准默认	最常见
027	rw-r----- (640)	rwxr-x--- (750)	组只读	项目协作
077	rw----- (600)	rw----- (700)	完全私有	高安全性需求
027	rw-r----- (640)	rwxr-x--- (750)	组协作	服务器常用

⚠ 重点: 记住022

SHELL

```
# RHEL默认umask值
root用户默认: 0022
普通用户默认: 0002

# 区别原因:
# root: 文件默认644 (其他人不能修改)
# user: 文件默认664 (同组用户可以修改)
```

umask实际操作示例:

```
# 示例1: 查看当前umask
[root@servera ~]# umask
0022

# 说明: 显示为八进制数
# 0022表示:
#   第1位(0): 特殊权限掩码
#   第2位(2): 所有者权限掩码 (不掩码)
#   第3位(2): 组权限掩码 (去除写权限)
#   第4位(2): 其他权限掩码 (去除写权限)

# 示例2: 测试umask效果
[root@servera ~]# umask
0022

[root@servera ~]# touch testfile.txt
[root@servera ~]# mkdir testdir
[root@servera ~]# ls -l testfile.txt
-rw-r--r--. 1 root root 0 Jan 15 10:00 testfile.txt
#           ↑
#         644权限

[root@servera ~]# ls -ld testdir
drwxr-xr-x. 2 root root 4096 Jan 15 10:00 testdir
#           ↑
#         755权限

# 示例3: 临时修改umask
[root@servera ~]# umask 077
[root@servera ~]# touch private.txt
[root@servera ~]# ls -l private.txt
-rw-----. 1 root root 0 Jan 15 10:01 private.txt
#           ↑
#         600权限 (完全私有)

# 示例4: 符号方式显示umask
[root@servera ~]# umask -S
u=rwx,g=rx,o=rx
# 以符号方式显示当前umask掩码
```

5.2 umask的永久设置

设置umask的作用域:

umask 设置层级

1. 全局设置（影响所有用户）
 - /etc/bashrc
 - /etc/profile
 - /etc/profile.d/*.sh
2. 用户设置（只影响特定用户）
 - ~/.bashrc
 - ~/.bash_profile
3. 临时设置（仅当前会话）
 - umask 022

方法1: 全局设置（影响所有用户）

SHELL

```
# 编辑全局bash配置文件
vim /etc/bashrc

# 在文件末尾添加：
umask 022

# 保存退出
# 所有新登录的用户都会使用这个umask值

# 或者在/etc/profile.d/中创建自定义脚本（推荐）
vim /etc/profile.d/custom-umask.sh

# 添加内容：
# 设置系统默认umask
umask 022

# 保存后，所有用户登录时都会执行此脚本
```

⚠ 修改全局配置的注意事项：

SHELL

```
# ⚠ 警告：修改/etc/bashrc会影响所有用户
# 包括系统服务和其他用户
# 建议：使用/etc/profile.d/创建独立脚本
# 好处：易于管理和删除

# 验证配置文件语法：
source /etc/bashrc
# 如果没有错误信息，说明语法正确
```

方法2: 用户设置（只影响特定用户）


```
# 编辑用户的bash配置文件
vim ~/.bashrc

# 在文件末尾添加:
umask 0077

# 保存退出
# 重新登录或执行:
source ~/.bashrc

# 验证:
[student@node1 ~]$ umask
0077
[student@node1 ~]$ touch test.txt
[student@node1 ~]$ ls -l test.txt
-rw-----. 1 student student 0 Jan 15 10:05 test.txt
#           ↑
#           600权限（只有用户本人可访问）
```

⚠ 易错点: **~/.bashrc vs ~/.bash_profile**

```
# ~/.bashrc
# - 每次打开交互式非登录shell时执行
# - 在图形界面中打开新终端时执行
# - 执行bash子shell时执行

# ~/.bash_profile
# - 仅在登录时执行一次
# - 通过SSH登录时执行
# - 通常会调用~/.bashrc

# 推荐做法:
# 在两个文件中都设置, 或者在.bash_profile中调用.bashrc
```

5.3 umask场景分析

场景1: 预测新文件的权限

```
# 题目：当前umask为027，创建新文件和目录的权限是什么？
# 分析：
# 文件：0666 - 027 = ?
#   666 = rw-rw-rw-
#   027 = ---wxrwx
# 计算：
#   所有者：6-0 = 6 (rw-)
#   组：      6-2 = 4 (r--)
#   其他：    6-7 = 0-1 = 问题！
#
# 正确计算（按位）：
#   所有者：r(4)+w(2)+x(1) - 0 = rwx = 7
#   组：      r(4)+w(2)+x(1) - 2(无x) = rw- = 6
#   其他：    r(4)+w(2)+x(1) - 7 = --- = 0
#
# 但文件不能有执行权限！
#   所有者：min(7, 6) = 6 (rw-)
#   组：      min(6, 4) = 4 (r--)
#   其他：    min(0, 0) = 0 (---)
# 结果：rw-r----- (640)
#
# 目录：0777 - 027 = 0750 (rwxr-x---)

# 快速记忆法：
# umask 027:
# 文件：640 (rw-r-----)
# 目录：750 (rwxr-x---)
```

场景2：设置安全的默认权限

```
# 题目：要求用户创建的文件默认只能本人访问
# 分析：需要文件权限为600 (rw-----)
# 计算：666 - umask = 600
#       umask = 666 - 600 = 066
# 答案：umask 0066

# 验证：
[root@servera ~]# umask 0066
[root@servera ~]# touch test.txt
[root@servera ~]# ls -l test.txt
-rw-----. 1 root root 0 Jan 15 10:10 test.txt
```

场景3：创建协作环境的默认权限

```
# 题目：多个用户在同一个项目组工作
# 要求：新文件默认组可读写，其他用户只读
# 分析：需要文件权限为664 (rw-rw-r--)
# 计算：666 - umask = 664
#          umask = 666 - 664 = 002
# 答案：umask 0002

# 验证：
[root@servera ~]# umask 0002
[root@servera ~]# touch project.txt
[root@servera ~]# ls -l project.txt
-rw-rw-r--. 1 root root 0 Jan 15 10:12 project.txt
#          ↑
#          664权限
```

⚠ 技巧：umask计算速查表

```
# 题目要求的文件权限 → 需要的umask值
# 777 (rwxrwxrwx) → 000 (极少用)
# 755 (rwxr-xr-x) → 022 (文件不适用，仅目录)
# 644 (rw-r--r--) → 022 (最常见)
# 640 (rw-r-----) → 027 (协作环境)
# 600 (rw-----) → 077 (高安全性)
# 750 (rwxr-x---) → 027 (目录)
# 700 (rwx-----) → 077 (私有目录)

# 记忆口诀：
# 标准环境：022
# 协作开发：002
# 高度安全：077
# 项目组：    027
```

演示：权限管理实践

演示目标

掌握文件权限和所有权的设置

演示步骤

步骤1：创建测试文件

```
# 创建测试目录和文件
mkdir /tmp/perms-test
touch /tmp/perms-test/file1.txt
touch /tmp/perms-test/file2.txt
```

步骤2：查看和修改权限

```
# 查看初始权限
ls -l /tmp/perms-test/

# 修改权限为644
chmod 644 /tmp/perms-test/file1.txt

# 为file2添加执行权限
chmod +x /tmp/perms-test/file2.txt

# 验证修改
ls -l /tmp/perms-test/
```

步骤3：修改所有权

```
# 修改所有者
chown student /tmp/perms-test/file1.txt

# 修改组
chown :wheel /tmp/perms-test/file2.txt

# 验证修改
ls -l /tmp/perms-test/
```

步骤4：设置特殊权限

```
# 创建协作目录
mkdir /tmp/shared
chown :wheel /tmp/shared

# 设置SGID权限
chmod 2770 /tmp/shared

# 验证设置
ls -ld /tmp/shared
# 输出应该显示: drwxrws---
```

总练习:

练习名称	对应章节	说明
总练习：控制对文件的访问	第07章	文件权限管理综合练习

下午 - 进程管理

第六部分：进程基础

课前故事：Unix进程管理的诞生

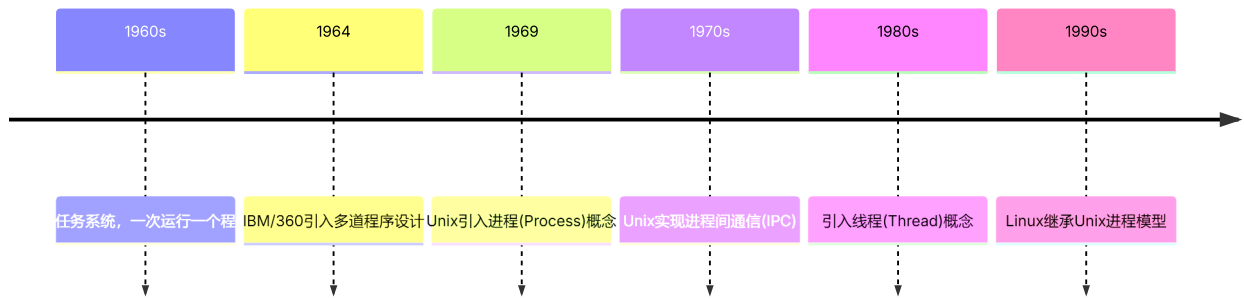
1960年代，在早期的计算机系统中，一次只能运行一个程序。程序员需要排队等待使用计算机，运行完自己的程序后，下一个程序员才能使用。

问题：

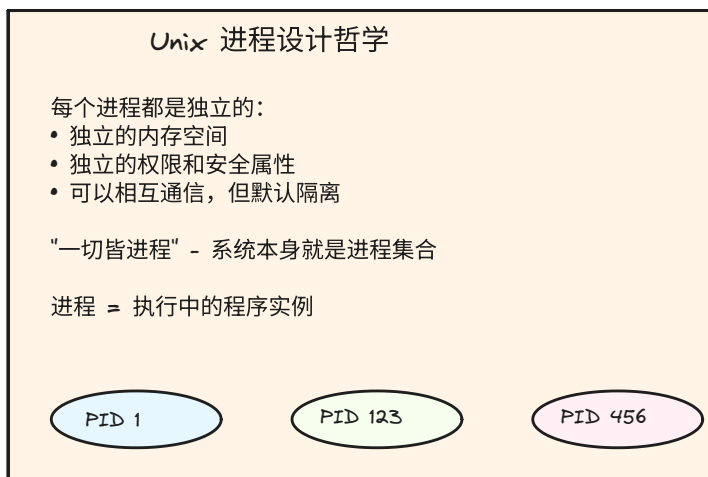
- 单任务系统的限制：
- 1. CPU利用率低 - 等待I/O时CPU空闲
 - 2. 响应时间长 - 用户需要等待很长时间
 - 3. 资源浪费 - 内存、外设无法充分利用

解决方案：多进程系统

进程管理发展简史



Unix进程模型的革命性：



⚠ 重要性：

- 进程管理是系统管理员的日常工作
- 实际工作中常要求查找和终止特定进程
- 理解进程状态对故障排查至关重要

6.1 进程概念深度解析

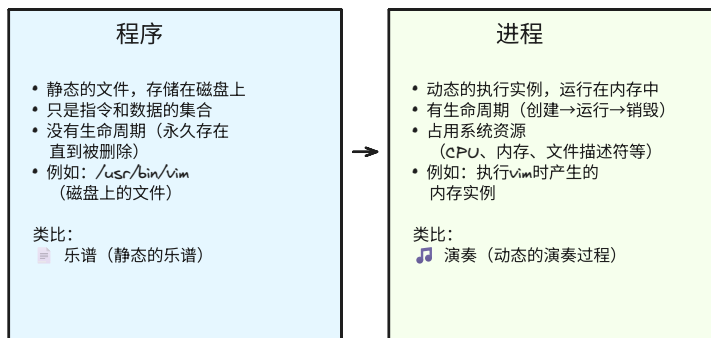
什么是进程？

进程（Process）是正在运行的程序实例，是操作系统进行资源分配和调度的基本单位。

进程包含的要素:

1. 程序代码 (代码段)
2. 程序数据 (数据段)
3. 进程控制块 (PCB)
 - 进程ID (PID)
 - 进程状态
 - CPU寄存器
 - 内存管理信息
 - I/O状态信息

进程与程序的区别 (重要概念):



- 同一乐谱可以同时被多个乐队演奏
- 同一程序可以同时运行多个进程

进程的实际示例:

示例1: 同一程序的多个进程

启动多个vim实例

```
vim file1.txt & # 进程1
vim file2.txt & # 进程2
vim file3.txt & # 进程3
```

查看: 同一个程序 (vim), 三个独立进程

```
ps aux | grep vim
```

```
# user  1234  0.1  0.2  12345  6789 pts/0  S   10:00   0:00 vim file1.txt
# user  1235  0.1  0.2  12345  6789 pts/0  S   10:01   0:00 vim file2.txt
# user  1236  0.1  0.2  12345  6789 pts/0  S   10:02   0:00 vim file3.txt
```

#

↑

不同的PID (进程ID)

示例2: 父子进程关系

当你运行bash脚本时

```
bash script.sh
```

进程树:

```
# systemd (PID 1)
```

```
#   └─ bash (你的登录shell)
```

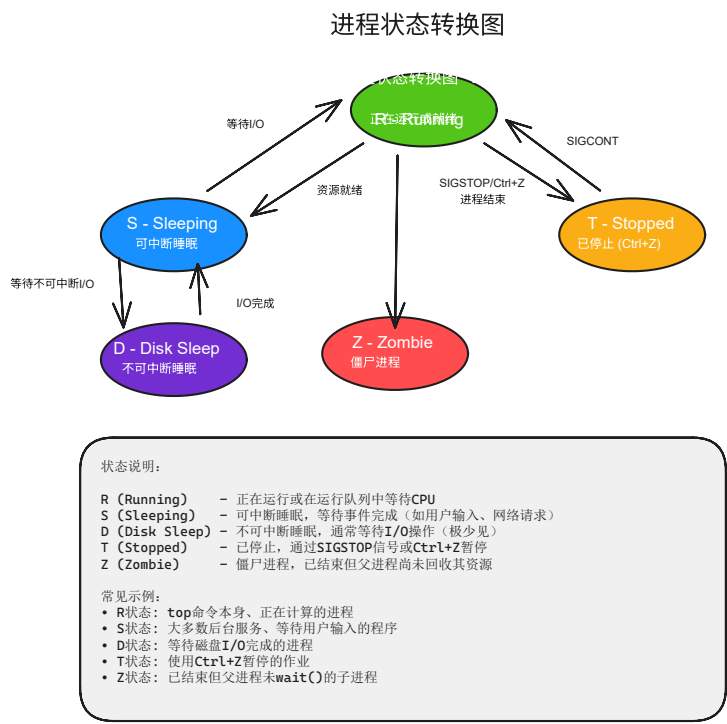
```
#       └─ bash (script.sh的子进程)
```

```
#           └─ 命令1 (script.sh中执行的命令)
```

```
#           └─ 命令2 (script.sh中执行的命令)
```

6.2 进程状态详解

进程生命周期（5种基本状态）：



进程状态详解：

状态	标志	名称	详细说明	常见示例	重要性
运行	R	Running	正在运行或在运行队列中等待	top 命令本身	☆☆☆
睡眠	S	Sleeping	可中断睡眠，等待事件完成	等待用户输入的进程	☆☆☆
不可中断	D	Disk sleep	不可中断睡眠，通常等待I/O	等待磁盘I/O的进程	☆☆
已停止	T	Stopped	已停止（通常通过信号）	被Ctrl+Z停止的进程	☆☆
僵停	Z	Zombie	僵停进程，已退出但父进程未回收	已退出但未被wait的进程	☆

⚠ 易错点：STAT列的附加标志


```
# ps输出中STAT列可能包含多个字符：
# 主要状态：
R  = 正在运行
S  = 睡眠
D  = 不可中断睡眠
T  = 已停止
Z  = 僵停

# 附加标志（重要！）：
s  = 会话领导者（session leader）
+  = 前台进程组
l  = 多线程进程
<  = 高优先级进程
N  = 低优先级进程
L  = 锁定在内存中

# 示例：
# Ss  = 睡眠 + 会话领导者（如bash）
# R+  = 运行 + 前台进程
# Sl  = 睡眠 + 多线程
# Ss+ = 睡眠 + 会话领导者 + 前台进程
```

进程状态转换实际示例：

```
# 场景1: 创建和运行进程
# 启动一个sleep进程
sleep 100 &
# 状态: S (可中断睡眠)
ps aux | grep sleep

# 场景2: 前台进程被挂起
vim test.txt
# 按Ctrl+z
# 状态从S变为T (停止)
jobs
# [1]+  Stopped  vim test.txt

# 场景3: 恢复后台进程
bg %1
# 状态从T变为S (睡眠, 后台运行)

# 场景4: 僵尸进程 (可能遇到)
# 父进程没有调用wait()回收子进程
# 子进程退出但PCB仍然存在
ps aux | grep Z
# 输出: Z+ (僵尸进程)

# 清理僵尸进程:
# 方法1: 重启父进程
kill -HUP <父进程PID>
# 方法2: 杀死父进程 (让init进程接管)
kill -9 <父进程PID>
```

⚠ 场景分析: 僵尸进程

```
# 题目：如何识别和清理僵尸进程？

# 识别僵尸进程：
ps aux | grep Z
# 或
ps -el | grep Z

# 输出示例：
# root    1234   0.0   0.0    0    0 ?        Z    10:00   0:00 [defunct]
#                               ↑
#                               僵尸进程标记

# 清理方法：
# 1. 找到僵尸进程的父进程
ps -eo pid,ppid,stat,comm | grep 1234
# 1234   1000   Z    defunct
#         ↑
#         父进程PID

# 2. 给父进程发送SIGHUP信号，让它回收子进程
kill -HUP 1000

# 3. 如果父进程无响应，终止父进程
kill -9 1000
# 僵尸进程会被init (PID 1) 接管并清理
```

相关练习：

练习名称	对应章节	说明
练习：CONTROLLING JOBS	第08章 第02节	控制作业（前台/后台/挂起/恢复）

第七部分：查看进程

ps命令的诞生

ps (process status) 是Unix最古老的命令之一，自1970年代以来一直是查看进程的主要工具。

为什么需要ps？

- 快速查看进程状态（快照）
- 查找特定进程
- 调试进程问题

- 监控系统资源使用

⚠️ 必须掌握：ps是最常用的进程管理命令！

7.1 ps命令完全指南

ps基础语法：

```
ps [选项]
```

SHELL

```
# ps有两种选项风格：
# 1. BSD风格（不带横杠）
# 2. UNIX风格（带横杠）
```

ps常用选项对比表：

风格	选项	含义	输出格式	常用度
BSD	aux	所有进程详细信息	宽格式	☆☆☆
UNIX	-ef	所有进程完整格式	标准格式	☆☆☆
BSD	auxf	树形显示进程	树形结构	☆☆
UNIX	-eLf	显示线程	包含线程	☆

ps aux - BSD风格详解（最重要）：

```
# 基本语法
```

```
ps aux
```

SHELL

```
# 参数分解：
```

```
a = all users（所有用户的进程）
```

```
u = user-oriented format（用户导向格式）
```

```
x = include processes without a controlling terminal（包括无控制终端的进程）
```

```
# 完整输出示例：
```

```
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1   0.0   0.3 193564   7608 ?        Ss   10:00   0:02
/usr/lib/systemd/systemd
root    456   0.0   0.1  12345   2345 ?        Ss   10:00   0:00 /usr/sbin/sshd -D
student  1234   0.5   1.2 345678 12345 pts/0    Ss   10:30   0:01  bash
student  5678   0.0   0.0  12345    678 pts/0    R+   10:35   0:00  ps aux
```

ps输出字段完整说明：

字段	全称	含义	示例值	说明
USER	-	进程所有者	root, student	启动进程的用户
PID	Process ID	进程ID	1234	唯一标识进程的数字
%CPU	-	CPU使用率	0.5	占用CPU的百分比
%MEM	-	内存使用率	1.2	占用内存的百分比
VSZ	Virtual Memory	虚拟内存大小	345678	占用的虚拟内存 (KB)
RSS	Resident Set Size	物理内存大小	12345	占用的物理内存 (KB)
TTY	Teletype	终端设备	pts/0, ?	进程关联的终端
STAT	Status	进程状态	Ss, R+, Z+	进程当前状态
START	-	启动时间	10:30, Jan15	进程启动的时间
TIME	-	累计CPU时间	0:01	进程占用的总CPU时间
COMMAND	-	命令名称	/bin/bash	执行的命令

 重点：TTY字段

SHELL

```
# TTY值含义：
?           = 无控制终端（守护进程）
pts/0       = 伪终端0（SSH或图形终端）
tty1        = 虚拟终端1（Ctrl+Alt+F1）
tty2        = 虚拟终端2（Ctrl+Alt+F2）

# 示例：
# systemd的TTY是 ?（守护进程，不需要终端）
# sshd的TTY是 ?（守护进程）
# bash的TTY是 pts/0（用户交互式shell）
```

ps实际应用示例（使用场景）：

场景1: 查看所有进程 (最常用)

```
ps aux
```

场景2: 查找特定进程 (必须掌握)

```
ps aux | grep nginx
```

```
ps aux | grep httpd
```

```
ps aux | grep sshd
```

输出示例:

```
# root 1234 0.0 0.2 12345 3456 ? Ss 10:00 0:00 nginx: master process
```

```
# nginx 1235 0.0 0.1 12789 2345 ? S 10:00 0:00 nginx: worker process
```

场景3: 查看特定用户的进程

```
ps -u student
```

```
ps -u root
```

场景4: 查看进程树 (了解进程关系)

```
ps auxf
```

或

```
ps -ef --forest
```

输出示例:

```
# PID TTY STAT TIME COMMAND
```

```
# 1 ? Ss 0:02 /usr/lib/systemd/systemd --switched-root --system -
```

```
# 456 ? Ss 0:00 \_ sshd: /usr/sbin/sshd -D [listener]
```

```
# 789 pts/0 Ss 0:00 \_ bash
```

```
# 1234 pts/0 R+ 0:00 \_ ps auxf
```

场景5: 按CPU使用率排序 (查找高CPU进程)

```
ps aux --sort=-%cpu | head -10
```

-%cpu = 降序排列 (从高到低)

+%cpu = 升序排列 (从低到高)

场景6: 按内存使用率排序 (查找高内存进程)

```
ps aux --sort=-%mem | head -10
```

场景7: 查看完整命令行 (参数被截断时)

```
ps aux | grep java
```

如果命令行太长被截断, 使用:

```
ps auxww | grep java
```

ww = 宽输出, 不截断命令行

场景8: 查看进程的父进程ID (PPID)

```
ps -eo pid,ppid,comm
```

-e = 所有进程

-o = 自定义输出格式

pid,ppid,comm = 显示PID、PPID和命令名

输出示例:

```
# PID PPID COMMAND
```

```
#      1      0 systemd
#    456      1 sshd
#    789    456 bash
#   1234    789 vim

# 场景9: 查看特定PID的详细信息
ps -fp 1234
# -f = full format (完整格式)
# -p = 指定PID
```

⚠ 技巧: **ps**查找进程的常用命令

```
# 最常用的5个ps命令:

# 1. 查找特定进程 (最考)
ps aux | grep processname

# 2. 查看所有进程
ps aux

# 3. 查看进程树
ps -ef --forest

# 4. 查找高CPU进程
ps aux --sort=-%cpu | head -10

# 5. 查看特定用户的进程
ps -u username
```

SHELL

ps - UNIX风格 (-ef选项) :

UNIX风格语法

`ps -ef`

参数分解:

-e = 显示所有进程

-f = full format (完整格式)

输出示例:

```

UID    PID  PPID  C  STIME TTY          TIME CMD
root     1      0  0  10:00 ?          00:00:02 /usr/lib/systemd/systemd
root   456      1  0  10:00 ?          00:00:00 /usr/sbin/sshd -D
student 789   456  0  10:30 pts/0    00:00:00 -bash

```

UNIX风格的字段:

UID = 用户ID (与USER相同)

PID = 进程ID

PPID = 父进程ID (重要!)

C = CPU使用率 (最近)

STIME = 启动时间

TTY = 终端

TIME = 累计CPU时间

CMD = 命令 (只显示命令名)

⚠️ ps aux vs ps -ef (如何选择)

ps aux (BSD风格)

优点:

- 显示CPU和内存百分比 (更直观)

- %CPU和%MEM便于判断资源占用

- COMMAND显示完整命令行

#

缺点:

- 不显示父进程ID (PPID)

#

适用场景:

- 查找高CPU/内存占用进程

- 一般性进程查看

#

#

ps -ef (UNIX风格)

优点:

- 显示PPID (父进程ID)

- 标准化输出, 便于脚本处理

#

缺点:

- C字段不如%CPU直观

#

适用场景:

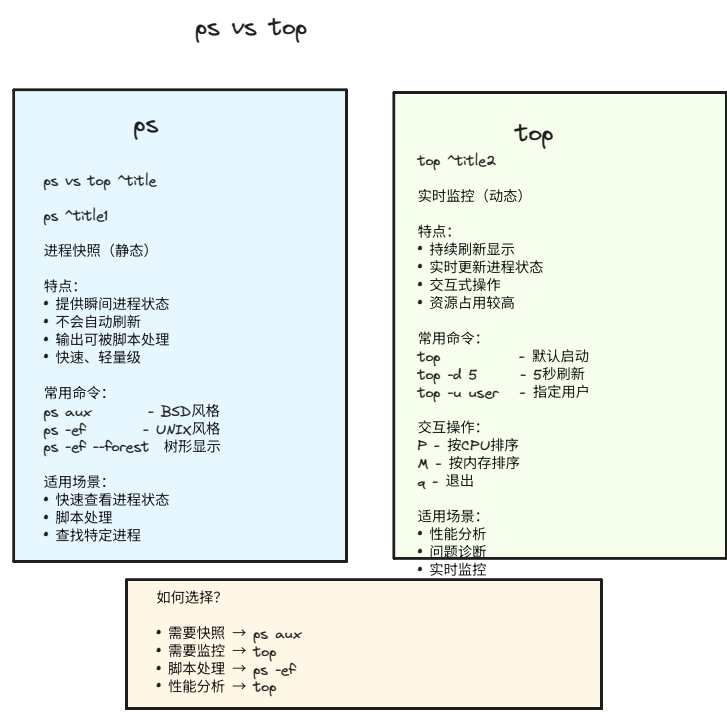
- 需要了解进程父子关系

- 编写脚本处理进程信息

7.2 top命令完全指南

top简介：

top是Linux系统中最常用的实时进程监控工具。与ps不同，top会动态刷新显示信息，是系统管理员进行性能分析的首选工具。



⚠ 注意：top提供实时监控，ps提供快照

top启动语法：

SHELL

```
# 基本语法
top [选项]

# 常用选项：
top -d 5      # 每5秒刷新一次（默认3秒）
top -n 10     # 刷新10次后退出
top -p 1234   # 只监控PID为1234的进程
top -u student # 只显示student用户的进程
top -b       # 批处理模式（适合脚本）
```

top输出完整解析：

```
# 启动top后的完整输出:
top - 10:30:45 up 2:30, 2 users, load average: 0.03, 0.05, 0.01
Tasks: 120 total, 1 running, 119 sleeping, 0 stopped, 0 zombie
%Cpu(s): 5.0 us, 2.0 sy, 0.0 ni, 92.0 id, 1.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem: 3840.0 total, 512.0 free, 2048.0 used, 1280.0 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used. 1792.0 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR S  %CPU  %MEM    TIME+  COMMAND
 1234 root        20   0  193564   7608  4567 S   0.0   0.3   0:02.12 systemd
 4567 root        20   0  123456   2345  1234 S   0.0   0.1   0:00.05 sshd
 8901 student    20   0  345678  12345  6789 R   0.5   1.2   0:01.23 firefox
```

top摘要信息（前5行）详解：

第1行：系统信息汇总

```
top - 10:30:45 up 2:30, 2 users, load average: 0.03, 0.05, 0.01

字段解析：
top - 10:30:45 = 当前时间
up 2:30        = 系统运行时间（2小时30分）
2 users        = 当前登录用户数
load average: 0.03, 0.05, 0.01 = 1分钟、5分钟、15分钟平均负载
```

⚠️ 重点：load average（平均负载）

```
# load average的含义:
# - 表示运行队列中的平均进程数
# - 三个数值: 1分钟、5分钟、15分钟
# - 负载 = CPU使用率 + 等待I/O的进程

# 如何判断负载是否过高?
# 单核CPU:
#   load < 1.0   = 正常
#   load > 1.0   = 过载
#   load > 2.0   = 严重过载
#
# 4核CPU:
#   load < 4.0   = 正常
#   load > 4.0   = 过载
#
# 公式: load average < CPU核心数 = 正常

# 查看CPU核心数:
lscpu | grep "^CPU(s)"
# 或
nproc

# 示例分析:
# load average: 2.00, 1.50, 1.00
# 在单核CPU上: 严重过载!
# 在4核CPU上: 负载正常
```

第2行: 进程统计

```
Tasks: 120 total, 1 running, 119 sleeping, 0 stopped, 0 zombie
```

字段解析:

```
120 total    = 进程总数
1 running   = 正在运行的进程
119 sleeping = 睡眠状态进程
0 stopped   = 已停止进程
0 zombie     = 僵尸进程 (⚠️ 如果这里有数字, 需要清理!)
```

⚠️ 警告: 僵尸进程

```
# 如果top显示:
Tasks: 120 total, 1 running, 118 sleeping, 0 stopped, 1 zombie
#                                     ↑
#                                     有1个僵尸进程!

# 处理方法:
# 1. 找到僵尸进程的PID (从进程列表)
# 2. 查找其父进程:
ps -eo pid,ppid,stat,comm | grep <僵尸PID>
# 3. 给父进程发送SIGHUP:
kill -HUP <父进程PID>
```

第3行: CPU使用情况

```
%Cpu(s):  5.0 us,  2.0 sy,  0.0 ni, 92.0 id,  1.0 wa,  0.0 hi,  0.0 si,  0.0
st
```

字段详解:

```
us (user)      = 用户空间程序占用CPU百分比 5.0%
sy (system)    = 内核空间占用CPU百分比 2.0%
ni (nice)      = ni值调整过的进程占用CPU 0.0%
id (idle)      = CPU空闲百分比 92.0%
wa (wait)      = 等待I/O的CPU时间百分比 1.0%
hi (hardware)  = 硬件中断占用CPU 0.0%
si (software)  = 软件中断占用CPU 0.0%
st (steal)     = 被hypervisor偷走的CPU时间(虚拟机) 0.0%
```

⚠ 性能分析技巧: CPU使用率

```
# 正常情况:
us + sy < 70%, id > 30% = CPU负载正常
wa > 20% = 磁盘I/O瓶颈 (需要检查磁盘性能)

# 问题诊断:
us很高 (>80%) = 用户程序占用大量CPU (可能是计算密集型程序)
sy很高 (>20%) = 内核占用大量CPU (可能是系统调用过多)
wa很高 (>20%) = I/O等待严重 (磁盘或网络瓶颈)
id很低 (<10%) = CPU几乎满载

# 示例分析:
# %Cpu(s): 85.0 us, 10.0 sy, 5.0 id
# → 用户进程占用85% CPU, 系统正常

# %Cpu(s): 30.0 us, 20.0 sy, 50.0 wa
# → 50%时间在等待I/O, 磁盘性能有问题!
```

第4行: 内存使用情况

MiB Mem: 3840.0 total, 512.0 free, 2048.0 used, 1280.0 buff/cache

字段详解:

total = 物理内存总量 3840 MB
 free = 完全空闲的内存 512 MB
 used = 应用程序使用的内存 2048 MB
 buff/cache = 缓冲区和缓存 1280 MB

重要: Linux内存使用哲学
 # "free内存"不是真实可用内存
 # buff/cache可以被回收使用
 # 真实可用内存 = free + buff/cache

⚠ 内存计算技巧 (常考)

计算实际可用内存:

可用内存 = free + buff/cache
 可用内存 = 512 + 1280 = 1792 MB

计算内存使用率:

使用率 = used / total × 100%
 使用率 = 2048 / 3840 × 100% = 53.3%

更准确的内存使用率:

实际使用率 = (total - free - buff/cache) / total × 100%
 实际使用率 = (3840 - 512 - 1280) / 3840 × 100% = 53.3%

第5行: 交换空间使用情况

MiB Swap: 2048.0 total, 2048.0 free, 0.0 used. 1792.0 avail Mem

字段详解:

total = 交换空间总量
 free = 空闲交换空间
 used = 已使用交换空间
 avail Mem = 可用内存 (估计值)

⚠ 警告: 如果Swap used > 0, 说明物理内存不足!

⚠ 性能问题: Swap使用

```
# 正常情况:
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used.
#                               ↑
#                               0 = 正常

# 问题情况:
MiB Swap: 2048.0 total, 1024.0 free, 1024.0 used.
#                               ↑
#                               使用了1024MB Swap!
# 说明: 物理内存不足, 系统正在使用磁盘作为内存
# 影响: 性能严重下降 (磁盘比内存慢1000倍)
# 解决: 增加物理内存或关闭占用内存的进程
```

top进程列表详解:

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1234	root	20	0	193564	7608	4567	S	0.0	0.3	0:02.12	systemd
4567	root	20	0	123456	2345	1234	S	0.0	0.1	0:00.05	sshd
8901	student	20	0	345678	12345	6789	R	0.5	1.2	0:01.23	firefox

字段说明:

- PID = 进程ID
- USER = 进程所有者
- PR = 优先级 (priority, 数值越小优先级越高)
- NI = nice值 (-20到19, 数值越小优先级越高)
- VIRT = 虚拟内存大小 (KB)
- RES = 物理内存大小 (KB)
- SHR = 共享内存大小 (KB)
- S = 进程状态 (R/S/D/T/Z)
- %CPU = CPU使用率
- %MEM = 内存使用率
- TIME+ = 累计CPU时间 (百分之一秒)
- COMMAND = 命令名

⚠ 内存字段详解 (重要!)

VIRT (Virtual Memory) = 虚拟内存

- 进程占用的虚拟内存总量
- 包括：代码段、数据段、共享库、栈、堆
- 包括：换页到Swap的内存
- 包括：尚未分配的内存（mmap的大文件）

RES (Resident Size) = 常驻内存 (RSS)

- 进程实际占用的物理内存
- 这是判断内存占用的关键指标
- 不包括换出到Swap的内存

SHR (Shared Memory) = 共享内存

- 进程使用的共享内存
- 包括：共享库、共享数据段
- SHR不一定是进程独占的

实际进程独占内存估算：

独占内存 = RES - SHR

示例分析：

VIRT=345678, RES=12345, SHR=6789

该进程独占内存 ≈ 12345 - 6789 = 5556 KB

top交互命令完整列表：

按键	功能	重要性
q	退出top	☆☆☆
Space	立即刷新	☆☆
h	显示帮助	☆
P	按CPU使用率排序（默认）	☆☆
M	按内存使用率排序	☆☆
T	按累计时间排序	☆
N	按PID排序	☆
k	终止进程（输入PID和信号）	☆☆☆
r	修改进程优先级	☆
d	修改刷新闻隔	☆
c	切换命令名显示（完整/简短）	☆
H	切换线程模式	☆
u	显示特定用户的进程	☆☆
W	保存当前配置	☆

场景1: 查找占用CPU最多的进程

1. 启动top
2. 按P键 (确保按CPU排序)
3. 查看第一行的进程

场景2: 查找占用内存最多的进程

1. 启动top
2. 按M键 (按内存排序)
3. 查看第一行的进程

场景3: 在top中终止进程

1. 启动top
2. 按k键
3. 输入要终止的PID: 1234
4. 输入信号: 9 (SIGKILL) 或15 (SIGTERM)

场景4: 查看特定用户的进程

1. 启动top
2. 按u键
3. 输入用户名: student
4. 只显示student用户的进程

场景5: 修改刷新频率

1. 启动top
2. 按d键
3. 输入新的刷新间隔 (秒): 5
4. 每5秒刷新一次

⚠️ 场景分析: 使用top诊断性能问题


```
# 题目：系统响应缓慢，使用top诊断问题

# 步骤1：启动top
top

# 步骤2：检查load average
# load average: 3.50, 3.00, 2.50
# → 如果是单核CPU，负载严重过高！
# → 如果是4核CPU，负载尚可接受

# 步骤3：检查CPU使用率
# %Cpu(s): 85.0 us, 10.0 sy,  5.0 id
# → 用户进程占用85% CPU
# → 按P键查看哪个进程占用CPU最多

# 步骤4：检查内存使用
# MiB Mem: 3840.0 total, 100.0 free, 3500.0 used, 240.0 buff/cache
# → 可用内存 = 100 + 240 = 340 MB (严重不足！)
# → 按M键查看哪个进程占用内存最多

# 步骤5：检查Swap使用
# MiB Swap: 2048.0 total, 1024.0 free, 1024.0 used
# → 使用了1024MB Swap! 物理内存严重不足
# → 需要增加内存或关闭占用内存的进程
```

相关练习：

练习名称	对应章节	说明
练习：MONITORING PROCESS ACTIVITY	第08章 第04节	监控进程活动 (ps/top)

第八部分：作业控制

8.1 前台和后台

后台运行进程：

在命令后添加&使进程在后台运行

```
sleep 1000 &
```

输出示例:

[1] 12345

说明:

[1] = 作业号

12345 = 进程ID(PID)

查看后台作业:

查看当前shell的后台作业

```
jobs
```

输出示例:

[1]+ Running sleep 1000 &

[2]- Running sleep 2000 &

将后台作业调到前台:

将作业1调到前台

```
fg %1
```

参数说明:

fg = foreground, 前台

%1 = 作业号

8.2 挂起和恢复

挂起前台进程:

按Ctrl+z挂起当前前台进程

进程会停止, 变为后台作业

恢复后台作业:

```
# 在后台继续运行挂起的作业
```

```
bg %1
```

```
# 参数说明:
```

```
# bg = background, 后台
```

```
# %1 = 作业号
```

将后台作业调到前台:

```
# 挂起: Ctrl+z
```

```
# 后台继续: bg %1
```

```
# 前台继续: fg %1
```

相关练习:

练习名称	对应章节	说明
练习: CONTROLLING JOBS	第08章 第02节	作业控制综合练习

第九部分：进程信号与终止

课前故事：Unix信号的诞生

1960年代，Unix设计师们需要一种机制来控制进程：既能优雅地通知进程终止，又能强制杀死无响应的进程。

问题背景:

如何让进程:

1. 优雅地关闭（保存数据、清理资源）
2. 立即停止（进程无响应时）
3. 暂停和恢复（调试和作业控制）
4. 重新加载配置（不中断服务）

解决方案：信号（**Signal**）机制

SIGHUP (1)

SIGINT (2)

SIGQUIT (3)

SIGKILL (9) ⚠️

SIGTERM (15)

SIGSTOP (19) ⚠️

常用信号详解:

SIGHUP (1)

- 重新加载配置 (常用)
- 终端断开时发送
- 可被捕获

SIGINT (2)

- Ctrl+C 发送此信号
- 请求中断进程
- 可被捕获

SIGKILL (9) ⚠️

- 强制终止进程
- 无法被捕获或忽略
- 立即终止, 无清理机会

SIGTERM (15)

- 优雅终止进程
- kill命令默认信号
- 可被捕获
- 进程可保存数据、清理资源

SIGSTOP (19) ⚠️

- 停止进程
- 无法被捕获或忽略
- Ctrl+Z 发送 SIGTSTP (20)

推荐做法:

- 先用 SIGTERM (kill 1234)
- 等待几秒
- 如无响应, 再用 SIGKILL (kill -9 1234)

Unix信号的智慧：

- 进程可以选择捕获、忽略或使用默认处理
- SIGKILL(9)和SIGSTOP(19)无法被捕获（强制执行）
- 提供了从"礼貌请求"到"强制执行"的多种方式

⚠️ 重要性：

- kill命令是最常用的进程管理命令
- 理解不同信号的作用对系统管理至关重要
- SIGHUP常用于重新加载配置（常见）

9.1 信号完全指南

什么是信号？

信号是Linux系统中进程间通信的一种机制，用于通知进程发生了某个事件。

信号的三个关键特性：

1. 异步：进程可以在任何时刻收到信号

2. 不可排队：相同信号不会排队，只保留最后一次

3. 可选择处理：进程可以捕获、忽略或使用默认动作

SHELL

标准信号完整列表（必记）：

信号	编号	默认动作	可捕获？	常见用途	重要性
SIGHUP	1	终止	✓	重新加载配置	☆☆☆
SIGINT	2	终止	✓	Ctrl+C中断	☆☆
SIGQUIT	3	终止+core	✓	Ctrl+\退出+转储	☆
SIGILL	4	终止+core	✓	非法指令	☆

信号	编号	默认动作	可捕获?	常见用途	重要性
SIGTRAP	5	终止+core	✓	断点陷阱	★
SIGABRT	6	终止+core	✓	abort()调用	★
SIGBUS	7	终止+core	✓	总线错误	★
SIGFPE	8	终止+core	✓	浮点异常	★
SIGKILL	9	终止	✗	强制终止	★★★
SIGUSR1	10	终止	✓	用户定义1	★
SIGSEGV	11	终止+core	✓	段错误	★
SIGUSR2	12	终止	✓	用户定义2	★
SIGPIPE	13	终止	✓	管道破裂	★
SIGALRM	14	终止	✓	定时器到期	★
SIGTERM	15	终止	✓	优雅终止	★★★
SIGSTKFLT	16	终止	✓	协栈错误	★
SIGCHLD	17	忽略	✓	子进程状态变化	★★
SIGCONT	18	继续	✓	继续已停止进程	★★
SIGSTOP	19	停止	✗	停止进程	★★
SIGTSTP	20	停止	✓	Ctrl+Z停止	★★
SIGTTIN	21	停止	✓	后台读	★
SIGTTOU	22	停止	✓	后台写	★

⚠️ 重点：不可捕获的信号

SHELL

```
# 只有2个信号无法被进程捕获或忽略：
1. SIGKILL (9) - 强制终止进程
2. SIGSTOP (19) - 强制停止进程

# 其他信号都可以被进程捕获或忽略
```

重要信号详解：

1. SIGHUP (1) - 挂起信号

```
# 用途：
# 1. 终止进程（原始用途）
# 2. 重新加载配置文件（现代用途，常考！）

# 示例1：让SSH重新加载配置
kill -1 $(cat /var/run/sshd.pid)
# 或
kill -HUP $(pgrep sshd)

# 示例2：让Nginx重新加载配置
kill -HUP $(cat /var/run/nginx.pid)
# nginx会重新读取配置文件，但不会中断服务

# 示例3：终端断开时，bash进程收到SIGHUP并终止
# 这就是为什么SSH断开后，运行的进程会被终止
# 解决方法：使用nohup或disown
```

⚠ 使用场景：SIGHUP重新加载配置

```
# 题目：修改了SSH配置，如何让配置生效但不中断服务？

# 错误做法：
systemctl restart sshd
# → 会中断所有SSH连接！

# 正确做法：
kill -HUP $(pgrep sshd)
# → SSH重新加载配置，不中断现有连接
```

2. SIGINT (2) - 中断信号

```
# 用途：请求进程中断（Ctrl+C发送SIGINT）

# 示例：
ping 8.8.8.8
# 按Ctrl+C
# ping进程收到SIGINT，优雅退出

# 与SIGTERM的区别：
# SIGINT = 交互式中断（用户主动）
# SIGTERM = 请求终止（程序或脚本）
```

3. SIGKILL (9) - 强制终止

用途：立即终止进程，无法被捕获或忽略

使用场景：

1. 进程无响应，SIGTERM无法终止
2. 僵尸进程（虽然效果有限）
3. 紧急情况，必须立即终止进程

示例：

```
kill -9 1234
```

或

```
kill -SIGKILL 1234
```

⚠ 危险：

- # - 进程没有机会保存数据
- # - 没有机会清理资源（临时文件、锁等）
- # - 可能导致数据不一致
- # - 只在必要时使用！

⚠ SIGTERM vs SIGKILL（必须掌握）

SIGTERM（15） - 优雅终止

优点：

- # - 进程可以捕获信号
- # - 可以保存数据、清理资源
- # - 可以关闭网络连接、释放文件锁
- # - 推荐首先使用

SIGKILL（9） - 强制终止

优点：

- # - 100%终止进程（进程无法拒绝）

缺点：

- # - 进程没有清理机会
- # - 可能导致数据丢失

推荐做法：

- # 1. 先用 SIGTERM (kill 1234)
- # 2. 等待几秒
- # 3. 如果无响应，再用 SIGKILL (kill -9 1234)

4. SIGTERM (15) - 优雅终止

用途：请求进程优雅退出（kill命令的默认信号）

示例：

```
kill 1234
```

等同于：

```
kill -15 1234
```

等同于：

```
kill -TERM 1234
```

进程收到SIGTERM后可以：

1. 保存数据到磁盘
2. 关闭网络连接
3. 释放文件锁和临时文件
4. 完成正在处理的任务
5. 优雅退出

5. SIGSTOP (19) 和 SIGCONT (18) - 停止和继续

SIGSTOP - 停止进程（无法被捕获）

```
kill -STOP 1234
```

进程暂停，等待SIGCONT

SIGCONT - 继续已停止的进程

```
kill -CONT 1234
```

进程从暂停状态恢复运行

与Ctrl+Z的区别：

Ctrl+Z 发送 SIGTSTP (20) - 可以被捕获

SIGSTOP (19) - 无法被捕获（强制停止）

示例：暂停和恢复进程

```
kill -STOP $(pgrep firefox)
```

Firefox暂停，不占用CPU

```
kill -CONT $(pgrep firefox)
```

Firefox恢复运行

6. SIGTSTP (20) - 终端停止

用途：Ctrl+Z发送此信号，停止前台进程

示例：

```
vim file.txt
```

按Ctrl+z

vim收到SIGTSTP，停止运行

作业进入后台（Stopped状态）

恢复：

```
fg %1 # 调到前台，发送SIGCONT
```


⚠️ 停止信号对比

SHELL

```
# SIGSTOP (19) - 强制停止
# - 无法被进程捕获或忽略
# - 总是有效
# - 示例: kill -STOP 1234

# SIGTSTP (20) - 终端停止
# - 可以被进程捕获或忽略
# - Ctrl+Z发送此信号
# - 示例: 按Ctrl+z

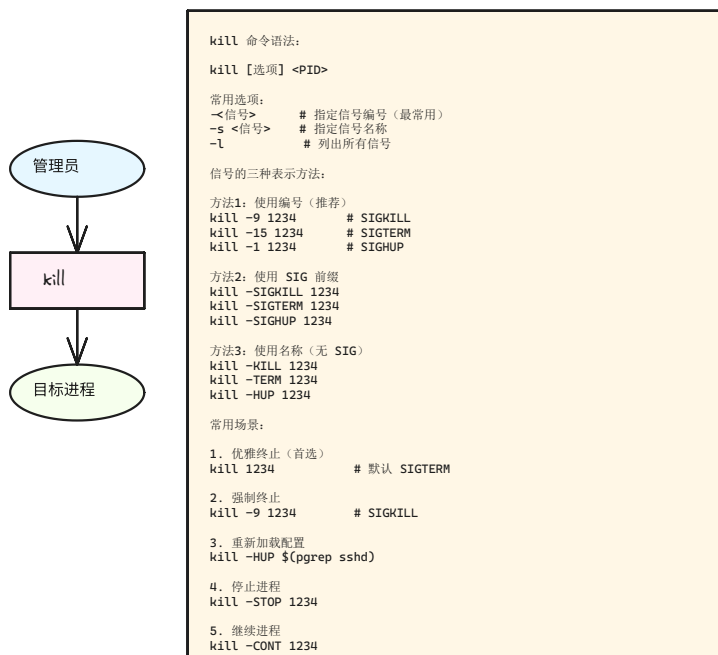
# 推荐做法:
# 正常使用: Ctrl+Z (SIGTSTP)
# 确保停止: kill -STOP (SIGSTOP)
```

9.2 kill命令完全指南

kill简介:

kill不是"杀死"进程, 而是"发送信号"给进程。默认发送SIGTERM(15), 请求进程优雅退出。

kill 命令: 发送信号



⚠️ 命名陷阱: kill不总是杀死进程!

SHELL

```
# kill的默认行为是"请求终止", 不是"强制杀死"
kill 1234 # 发送SIGTERM, 请求进程优雅退出
kill -9 1234 # 发送SIGKILL, 强制杀死进程
```

kill完整语法：

SHELL

```
kill [选项] <PID>
```

常用选项：

```
-s <信号>      # 指定信号名称或编号
-<信号>        # 指定信号编号（简写）
-l            # 列出所有信号
-<信号>        # 信号前加横杠（最常用）
```

kill使用示例（使用场景）：

SHELL

场景1：优雅终止进程（首选方法）

```
kill 1234
```

发送SIGTERM(15)，进程可以优雅退出

验证进程是否终止：

```
ps -p 1234
```

如果进程不存在，说明已终止

如果进程仍在运行，需要强制终止

场景2：强制终止进程

```
kill -9 1234
```

或

```
kill -SIGKILL 1234
```

发送SIGKILL(9)，立即终止进程

场景3：重新加载配置文件（常考）

```
kill -HUP $(pgrep sshd)
```

或

```
kill -1 $(cat /var/run/sshd.pid)
```

SSH重新加载配置，不中断现有连接

场景4：停止进程

```
kill -STOP 1234
```

进程暂停，不占用CPU

场景5：继续已停止的进程

```
kill -CONT 1234
```

进程从暂停状态恢复

场景6：查看所有信号

```
kill -l
```

输出：

```
# 1) SIGHUP      2) SIGINT    3) SIGQUIT  4) SIGILL    5) SIGTRAP
```

```
# ...
```

```
# 64) SIGRTMAX
```

⚠ 技巧：kill信号的三种表示方法

```
# 方法1: 使用信号编号 (最常用)
kill -9 1234      # SIGKILL
kill -15 1234     # SIGTERM
kill -1 1234      # SIGHUP

# 方法2: 使用信号名称 (带SIG前缀)
kill -SIGKILL 1234
kill -SIGTERM 1234
kill -SIGHUP 1234

# 方法3: 使用信号名称 (不带SIG前缀)
kill -KILL 1234
kill -TERM 1234
kill -HUP 1234

# 推荐: 方法1 (使用编号)
kill -9 1234
# 简短、快速、不易出错
```

⚠ 易错点: kill和killall的区别

```
# kill - 需要PID
kill 1234      # √ 正确
kill firefox  # X 错误! 需要PID, 不是进程名

# killall - 需要进程名
killall firefox # √ 正确
killall 1234    # 可能有问题 (如果1234不是进程名)

# 如果你想按名称终止进程, 使用:
pkill firefox  # 推荐
killall firefox # 也可以
```

9.3 killall和pkill完全指南

killall vs pkill vs pgrep:

命令	功能	参数	重要性
kill	按PID终止进程	PID	☆☆☆
killall	按名称终止所有匹配进程	进程名	☆☆
pkill	按模式匹配终止进程	模式	☆☆
pgrep	按模式查找进程PID	模式	☆☆

killall - 按名称终止所有进程:

```
# 基本语法
killall [选项] <进程名>

# 常用选项:
-9      # 发送SIGKILL
-15     # 发送SIGTERM (默认)
-u <用户> # 只终止特定用户的进程
-i      # 终止前询问
-w      # 等待所有进程终止

# 示例1: 终止所有sleep进程
killall sleep
# 终止所有名为sleep的进程

# 示例2: 强制终止所有firefox进程
killall -9 firefox
# 发送SIGKILL, 强制终止

# 示例3: 终止特定用户的进程
killall -9 -u student firefox
# 只终止student用户的firefox进程

# 示例4: 终止前询问
killall -i firefox
# 每个firefox进程都会询问是否终止

# 示例5: 等待进程终止
killall -w firefox
# 等待所有firefox进程终止后返回
```

⚠ killall危险警告!

```
# ⚠ 危险操作:
killall bash
# → 会终止你的shell! 你会立即退出登录!

killall python
# → 终止所有python进程, 包括系统服务!

# 安全建议:
# 1. 使用pkill (更精确的匹配)
# 2. 先用pgrep确认要终止的进程
# 3. 使用-i选项交互式终止
```

pkill - 按模式匹配终止进程:

```
# 基本语法
pkill [选项] <模式>

# 常用选项:
-9      # 发送SIGKILL
-u <用户> # 只终止特定用户的进程
-t <终端> # 只终止特定终端的进程
-f      # 匹配完整命令行
-n      # 只终止最新的进程
-o      # 只终止最老的进程

# 示例1: 终止所有firefox进程
pkill firefox
# 匹配进程名包含"firefox"的所有进程

# 示例2: 终止特定用户的进程
pkill -u student firefox
# 只终止student用户的firefox

# 示例3: 按终端终止进程
pkill -t pts/0
# 终止pts/0终端上的所有进程

# 示例4: 终止所有nginx进程 (包括worker)
pkill -9 -f nginx
# -f 表示匹配完整命令行, 不只是进程名

# 示例5: 只终止最新的进程
pkill -n sleep
# 只终止启动最晚的sleep进程

# 示例6: 只终止最老的进程
pkill -o sleep
# 只终止启动最早的sleep进程
```

pgrep - 查找进程PID:

```
# 基本语法
pgrep [选项] <模式>

# 常用选项:
-l          # 同时显示进程名
-f          # 匹配完整命令行
-u <用户>    # 只显示特定用户的进程
-t <终端>    # 只显示特定终端的进程
-n          # 只显示最新的进程PID
-o          # 只显示最老的进程PID

# 示例1: 查找进程PID
pgrep sshd
# 输出:
# 1234
# 5678

# 示例2: 显示进程名和PID
pgrep -l sshd
# 输出:
# 1234 sshd
# 5678 sshd

# 示例3: 查找特定用户的进程
pgrep -lu student
# -l = 显示进程名
# -u = 只显示student用户的进程
# 输出:
# 1234 bash
# 5678 firefox

# 示例4: 按终端查找进程
pgrep -t pts/0
# 显示pts/0终端的所有进程PID

# 示例5: 组合使用 (最实用)
pgrep -l -u student firefox
# 查找student用户的firefox进程, 显示进程名
# 输出:
# 7890 firefox
```

⚠ 技巧: **pgrep + kill**组合

场景：需要终止某个进程，但不知道PID

方法1：使用pgrep查找，然后kill

```
pgrep -l firefox # 找到PID: 7890
```

```
kill 7890 # 终止进程
```

方法2：一行命令（推荐）

```
kill $(pgrep firefox)
```

或

```
kill -9 $(pgrep firefox)
```

方法3：使用pkill（最简单）

```
pkill firefox
```

或

```
pkill -9 firefox
```

⚠ 使用场景：选择合适的终止命令

场景1：知道PID，需要优雅终止

```
kill 1234
```

场景2：知道PID，进程无响应

```
kill -9 1234
```

场景3：不知道PID，知道进程名

```
pkill firefox # 推荐
```

```
killall firefox # 也可以
```

场景4：需要终止特定用户的进程

```
pkill -u student firefox
```

场景5：需要重新加载配置

```
kill -HUP $(pgrep sshd)
```

场景6：需要先确认再终止

```
pgrep -l firefox # 确认进程
```

```
kill $(pgrep firefox) # 终止进程
```

演示：进程管理实践

演示目标

掌握进程监控和管理的基本操作

演示步骤

步骤1：查看进程

```
# 查看所有进程
ps aux

# 查看特定用户的进程
ps -u student

# 使用top实时监控
top

# 按q退出
```

SHELL

步骤2：作业控制

```
# 在后台运行sleep命令
sleep 500 &

# 查看后台作业
jobs

# 将作业调到前台
fg %1

# 按Ctrl+z挂起
# 输入bg在后台继续
bg %1
```

SHELL

步骤3：终止进程

```
# 查找sleep进程的PID
pgrep sleep

# 使用SIGTERM终止
kill 1234

# 如果无法终止，使用SIGKILL
kill -9 1234
```

SHELL

练习名称	对应章节	说明
练习：KILLING PROCESSES	第08章 第03节	终止进程 (kill/killall/pkill)
总练习：MONITORING AND MANAGING LINUX PROCESSES	第08章	进程管理综合练习

本日总结

关键命令速查

命令	作用	常用选项
chmod	修改权限	644, u+x, -R
chown	修改所有者	user:group, -R
chgrp	修改组	无
umask	查看设置默认权限	022, 077
ps	查看进程快照	aux, -ef
top	实时监控进程	无
kill	终止进程	-9, -15
killall	按名称终止	-9
pkill	按模式终止	-u user
pgrep	查找进程ID	-l
jobs	查看后台作业	无
fg	前台作业	%1
bg	后台作业	%1